Software Requirements Specification Template

The following annotated template shall be used to complete the Software Requirements Specification (SRS). The instructor must approve any modifications to the overall structure of this document.

Template Usage:
Text contained within angle brackets ('<', '>') shall be replaced by your project-specific information and/or details. For example, <Project Name> will be replaced with either 'Smart Home' or 'Sensor Network'.

Italicized text is included to briefly annotate the purpose of each section within this template. This text should not appear in the final version of your submitted SRS.

This cover page is not a part of the final template and should be removed before your SRS is submitted.

Acknowledgements:
Sections of this document are based upon the IEEE Guide to Software Requirements Specification (ANSI/IEEE Std. 830-1984).

Pitch Deck

Software Requirements Specification

Version 0.1

1/28/2026

Sophia Campola – Team Leader
Carlos Parra Camacho – Developer, Design
Clayton Smoker – Developer, Tester
Alyssa Young – Developer, Programmer

Pitch Deck

Revision History

Date	Description	Author	Comments
4/2/2026	Pitch Deck 0.2	Alyssa Young	Added another functional requirement based on the necessity of implementing basic AI bot logic for the computer-generated players to (1) be able to participate in the game and (2) have a challenging aspect to them.

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
Sophia Campola	Sophia Campola	Team Leader	4/3/2026
Carlos Parra Camacho	Carlos Parra Camacho	Developer, Designer	4/3/2026
Clayton Smoker	Clayton Smoker	Developer, Tester	4/3/2026

Table of Contents

1. INTRODUCTION **1**

1.1 PURPOSE 1
1.2 SCOPE 1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 1
1.4 OVERVIEW 1

2. GENERAL DESCRIPTION **1**

2.1 PRODUCT PERSPECTIVE 1
2.2 PRODUCT FUNCTIONS 2
2.3 USERS AND CHARACTERISTICS 2
2.4 GENERAL CONSTRAINTS 2
2.5 ASSUMPTIONS AND DEPENDENCIES 2
2.6 OPERATING ENVIRONMENT 2

3. SPECIFIC REQUIREMENTS **2**

3.1 EXTERNAL INTERFACE REQUIREMENTS 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 FUNCTIONAL REQUIREMENTS 3
3.2.1 <Functional Requirement or Feature #1> *3*
3.2.2 <Functional Requirement or Feature #2> *3*
3.3 USE CASES 4
3.3.1 Use Case #1 ..... *4*
3.3.2 Use Case #2 ..... *4*
3.4 NON-FUNCTIONAL REQUIREMENTS 4
3.5.1 Performance *5*
3.5.2 Reliability *5*
3.5.3 Availability *5*
3.5.4 Security *5*
3.5.5 Maintainability *5*
3.5.6 Portability *5*
3.5 DESIGN CONSTRAINTS 5
3.6 LOGICAL DATABASE REQUIREMENTS 5
3.7 OTHER REQUIREMENTS 5

4. ANALYSIS MODELS **5**

4.1 SEQUENCE DIAGRAMS 6

5. CHANGE MANAGEMENT PROCESS **6**

A. APPENDICES **6**

A.1 APPENDIX 1 7
A.2 APPENDIX 2 7

Figure 1: Data Flow Diagram Example 1 14
Figure 2 Data Flow Diagram Example 2 14

LIST OF FIGURES

LIST OF TABLES

Table 1: Use case 1 9
Table 2: Use case 2 9
Table 3: Use case 3 10

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide stakeholders and clients with a clear reference to the updated requirements that the project will meet. This document helps reduce misunderstandings by clearly outlining the system's design, implementation considerations, functionality, and constraints.

1.2 Scope

(1) - The software product to be produced is a serverless, web-based application titled **Pitch**. It is designed as a browser-based, client-side application compatible with desktop, tablet, and mobile devices, requiring no server-side infrastructure.
(2) - The **Pitch** application will provide a responsive, single-player experience where the user competes against rule-based computer bots that simulate opponents. The software will implement the core mechanics of the traditional game, including scorekeeping, bidding, dealing cards, and choosing the trump suit. Specifically, the application will enforce custom rules such as a minimum starting bid of 18 and a "throw away" feature for non-trump cards. It will also allow users to customize game settings, such as the winning score limit and bidding amounts, and will include a static "How to Play" section to educate users on these rules. Game data will be managed via local storage.

Conversely, the application will not support online multiplayer networking, as it is designed strictly for solo play. It will not include any user authentication or login systems, nor will it utilize external databases for long-term data storage. Finally, the project will not include an interactive "tutorial mode" or multiple rule variations of the game beyond the specific version defined.

(3) – The benefits this product provides are that it eliminates manual setup of the physical card game, reduces rule ambiguity by enforcing a consistent rule set, enhances accessibility by allowing users to play at any time, and improves user engagement through digital feedback. The objectives of this product are to accurately implement and enforce all game rules, provide a responsive and intuitive user interface, support configurable gameplay parameters, and to ensure consistent performance with game actions. The goals of this product are to deliver turn-based gameplay with automatic validation of legal moves and immediate feedback for invalid options and generating real-time score tracking and end-of-game summaries, including win/loss records.

1.3 Definitions, Acronyms, and Abbreviations

Client-side Application: The web-based application frontend designed for desktop browsers
Local Storage: Refers to the SQLite database stored on the computer, used for caching user data offline
Latency: The time taken for the server to respond to a request.
UI: user interface; the space where users interact with the digital product
Uptime: The percentage of time a system is fully operational and accessible to users, indicating its reliability
Script Injection: A type of cyber-attack where an attacker injects malicious scripts into a web application

1.4 Overview

The Software Requirements Specification (SRS) document lists and explains all the requirements, both functional and non-functional, that comprise the Pitch Deck application. This document is broken down into 5 general sections, each encompassing a different aspect of the system:
(1) Section 1 is an introductory section that provides an overview of the SRS document as a whole as well as important information about the Pitch Deck application itself such as its purpose and scope.

(2) Section 2 is a descriptive section that provides an overview of the application system such as the game's description, functions, constraints, assumptions, and dependencies to name a few.
(3) Section 3 describes the requirements section that details the application's software design, implementation, and testing including information pertaining to the functional and non-function requirements of the system as well as various use cases tested.
(4) Section 4 describes the analysis models used to develop the requirements for the system along with the type of sequence diagram used and the development description.
(5) Section 5 describes the change management process used to update the SRS document whenever the details of the project change, like a change in the system's requirements.

2. General Description

2.1 Product Perspective

The product, Pitch Deck, described in the SRS, is a new, self-contained product based on the card game Pitch. Our team's variation will have modified rules applicable to game play and scoring.

In the original game of Pitch, each round, every player can earn up to 4 points: game, high, low, and jack of trump suit. In Pitch Deck, the biding is expanded to 18, as the 5 and 9 of the trump are in play. Additionally, a throw in will be introduced after the bid, allowing players to get rid of their non-trump cards in hopes to gain more.

These alterations change the strategy needed to play the game and score logic. Although this digital game is a follow-on member of the original game Pitch, it is a stand-alone digital system, that will not be replacing any pervious software. Our team will build a web-based system with a user interface and a game server to implement rules.

2.2 Product Functions

Gameplay

- After each player receives 6 cards, player to left of dealer starts bid
- Winner of bid choices trump suit
- Supports throw-in
- Makes players follow suit, unless player does not have suit or throwing trump

Scoring

- If user wins the high in trump suit, they receive a point
- If user wins the low in trump suit, they receive a point
- If user wins the jack in trump suit, they receive a point
- If user wins the 5 in trump suit, they receive 5 points
- If user wins the 9 in trump suit, they receive 9 points
- If user wins game, they receive a point
    - To determine game point, you must count how many face/10 cards you have; whoever has the most points wins
        - An Ace is 4 points
        - A King is 3 points
        - A Queen is 2 points
        - A Jack is 1 point
        - A 10 is 10 points

- If user does not make their bid, the must subtract that number from their score
- If user makes bid, they will add points earned to their score

User-interface

- Allows user to choose what they want to play to (ex: 50, 75, 100)
- Notifies user when rules are being violated
- Displays users' cards

2.3 Users and Characteristics

Pitch Deck will be designed as a web browser-based card game with relatively simple and intuitive controls. The users that will engage with this project will be players with a wide range of technical expertise and gaming experience. Considering our product will not have a login system and, therefore, no ranking system, there will be no differentiation between frequent and/or experienced users and occasional and/or inexperienced users.
The classification of users can be divided into three categories:
(1) New players are users that have never played the card game Pitch before or are unfamiliar with playing a card game online. These players are expected not to know the rules and/or not know how to navigate the user interface and how to use the various features. Therefore, these players need instructions on how to play and simple mechanics to easily operate.
(2) Casual players are users that have played the card game Pitch before or are familiar with playing a card game online. These players are expected to know the rules and know how to navigate the user interface, but perhaps they won't know the variations to the rules or how to use all the features. Therefore, these players need a rundown of any new rules and explicit features with simple controls and minimal set-up.
(3) Experienced players are users that have played our version of the card game Pitch before and are very familiar with playing a card game online. These players are expected to know all the rules and know how to navigate the user interface and how to use all the provided features. Therefore, these players need adjustable settings with customized gameplay in order to keep the game fresh and detailed scoring and scoreboards.

2.4 General Constraints

(1) This product will be created through the use of basic web technologies and by a team of beginner developers. This will limit the use of advanced frameworks or features requiring complicated programming techniques.
(2) The development of this product must be completed within the span of approximately 3 months, which includes all the necessary documentation before the actual implementation. This will limit the amount of time available for design, testing, and any further expansion.
(3) This product will be created through a development team. This means that development is dependent on the availability and schedules of the team members, which may limit time for development and testing.
(4) This product will run on a web browser. This will limit access to any device-specific features and complicate the graphics and capabilities of the user interface on different devices.
(5) This product will function without requiring users to create accounts or log in. This will limit personalization, not allow for online multiplayer games, and will have no databases for persistent data storage for long-term data.
(6) The additional players for each game will be computer-controlled bots which will have simple, rule-based behavior. This will limit any advanced decision-making by the bots, which leads to a lack of complex gameplay.

2.5 Assumptions and Dependencies

(1) Modern Browser Environment: It is assumed that the end-users will access the application via a modern web browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) that supports HTML5, CSS3, and JavaScript.

Since the application is purely client-side, reliance on modern browser engines is critical. If a significant portion of the user base utilizes outdated browsers, the requirements for UI rendering and game logic execution would need to be altered to support legacy systems.

(2) Local Storage Availability: The system design assumes that the user's browser permits the use of Local Storage. This is necessary to fulfill requirements related to game state management and user customization. If Local Storage is disabled by the user (e.g., strictly private browsing modes or security policies), the requirement to maintain game settings across page reloads cannot be met without introducing a backend database.

(3) Device Computational Power: As the application relies on client-side execution for the rule-based computer bots, it is assumed that the user's device possesses sufficient processing power to handle the game logic and AI calculations without significant latency.

(4) Single-Device Persistence: Based on the "No Long-term Storage" assumption from the Project Charter, it is assumed that users do not require cross-device data synchronization. The requirements are scoped to single-session or single-device play. If this assumption changes (i.e., users require their statistics to be saved across different devices), the project scope would fundamentally change to require server-side infrastructure and user authentication.

(5) Team Availability Dependency: The project schedule and scope are dependent on the availability of the development team members. As noted in the Project Charter, the limited timeline (3 months) and the academic commitments of the team members are factors that may restrict the implementation of non-critical features.

2.6 Operating Environment

The product will support Windows 10 or later and macOS 12 or later. The product requires a basic hardware configuration, including a modern processor, at least 4 GB of RAM, and a display capable of standard screen resolution. User input will be provided through a keyboard and mouse. The software will run as a standalone application and is designed to peacefully coexist with other commonly used applications. It will not interfere with other software running on the device. The application will operate over standard internet connections.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

3.1.2 Hardware Interfaces

3.1.3 Software Interfaces

3.1.4 Communications Interfaces

3.2 Functional Requirements

3.2.1 Choose Card

3.2.1.1 Description and Priority

The Card Selection feature allows the user to interact with the game interface to select and play a specific card from their hand during their turn. This is the core mechanic of the gameplay loop.
Priority: High. Without this feature, the game is unplayable.

3.2.1.2 Stimulus/Response Sequences

Stimulus: The game engine determines it is the user's turn and unlocks the user's hand interface.
Response: The system highlights the user's cards and starts a turn timer (if applicable)
Stimulus: The user clicks on a specific card in their hand.

Response: The system validates if the card follows the rules (e.g., following the lead suit).
If valid: The card is animated moving from the hand to the center of the table. The user's turn end.
If invalid: The system displays a "Must follow suit" warning and the card remains in the hand.

3.2.1.3 Functional Requirements

REQ-1: As a user, I will be able to play a chosen card each round.
FR-01: The system shall verify that the selected card adheres to the game rules (lead suit constraints) before accepting the input.

3.2.2 Choose Bid

3.2.2.1 Description and Priority

The Bid Choosing feature presents the user with an interface at the start of the round to declare the number of points they expect to win. This determines the "High Bidder" who controls the trump suit.
Priority: High. This feature initiates the logic for the entire round.

3.2.2.2 Stimulus/Response Sequences

Stimulus: The dealing phase concludes.
Response: The system displays a modal or overlay with bidding options ranging from "Pass" to "18"

Stimulus: The user selects a bid value (e.g., "4").
Response: The system checks if the bid is higher than the current highest bid from previous bots.

3.2.2.3 Functional Requirements

REQ-2: As a user, I will be able to initially choose a bidding amount from 0-18 points.
FR-02: The system shall prevent the user from selecting a bid lower than the current highest bid unless they choose to "Pass" (0 points).

3.2.3 Choose Suit

3.2.3.1 Description and Priority

If the user wins the bidding phase, this feature allows them to determine the Trump suit (Hearts, Diamonds, Clubs, Spades) for the current round.

Priority: High. Essential for scoring and gameplay strategy.

3.2.3.2 Stimulus/Response Sequences

Stimulus: The bidding phase ends, and the user is identified as the High Bidder.
Response: The system prompts the user with a UI to select one of the four suits.

Stimulus: The user clicks on a suit icon (e.g., Spades).
Response: The system sets "Spades" as the Trump suit for the round, updates the UI to show the current Trump, and proceeds to the discard phase or the first trick.

3.2.3.3 Functional Requirements

REQ 3 – As a user, if I win the bid, I'm able to choose the trump suit for that round.
FR-03 - The system must restrict this feature only to the player who won the bid; other players (bots) will skip this step.

3.2.4 Choose Max Points Per Game

3.2.4.1 Description and Priority

This feature will allow the user to choose the total number of points the game will be played too. There should be options for 50, 75, 100, or 150. Although this feature is important, it is not necessary making it of low

priority. The benefit of having this functionality would be to allow them to choose the length of the game. This could be beneficial for them is they are in a time crunch.

- Benefit: 7
- Penalty: 2
- Cost: 6
- Risk: 3

3.2.4.2 Stimulus/Response Sequences

Stimulus: The user chooses the number the game will be played to.

Response: Once a player reaches the chosen number of game points, the game is completed and that player is declared the winner.

3.2.4.3 Functional Requirements

REQ 1 – This will allow the user to choose the total number of points the game will go to, before the game starts.

REQ 2- The feature will prevent the user from changing the game points after the game has started.

REQ 3 – The requirement is required to confirm the total game points before it is submitted.

REQ 4 – If the user tries to start the game without choosing a point total, the system will result in an error. This will then make the user choose a point total.

3.2.5 View Scoreboard

3.2.5.1 Description and Priority

This feature will allow the user to view the scoreboard at any point in the game. This will be updated after every Round. This functionality is of low priority, because it does not deal with the game's functionality/rules.

- Benefit: 9
- Penalty: 5
- Cost: 6
- Risk: 3

3.2.5.2 Stimulus/Response Sequences

Stimulus: The user wants to view the scoreboard before/during a game.

Response: The scoreboard should always appear on the top left-hand corner with the most recent up-to-date scores.

3.2.5.3 Functional Requirements

REQ 1 – This allows the user to view the score at any time.

REQ 2 – The scoreboard will always appear on the upper left-hand corner.

REQ 3 – The scoreboard will be updated after every round.

REQ 4 – The system will prevent the user from editing the score.

REQ 5 – The scoreboard will show the points each player has.

3.2.6 Card Lock

3.2.6.1 Description and Priority

The system will not allow users to discard or play invalid cards when it is their turn. For example, during the throw in the player is only allowed to discard non-trump cards. The system will prevent the player from discarding any trump cards. Since this functionality is crucial to the game, it is of high priority.

- Benefit: 9
- Penalty: 9
- Cost: 8

- Risk: 6

3.2.6.2 Stimulus/Response Sequences

Stimulus: The trump suit has been chosen by the highest bidder, so it is time to discard.

Response: The system will highlight the cards the player is allowed to discard (non-trump suit). They can discard as many of these cards as they would like.

Stimulus: Within each game round, there are sub-rounds. During the sub-rounds the winner of the bid or the previous sub-round will be throughout any card of their choosing.

Response: Once the first card of the sub-round is thrown, the user must follow suit or throw a trump card. The system will show what cards they are allowed to throw.

Stimulus: It is currently other players turn to throw a card.

Response: The system will not allow you to through a card out of turn.

3.2.6.3 Functional Requirements

REQ 1 – This helps users follow the rules of the game, ensuring no cheating.
REQ 2 – The system will show the user which cards they are allowed to play during each sub-round.
REQ 3 – The system will not allow users to throw a card when it is not their turn.
REQ 4 – During the throw in, the system will show the user which cards it is allowed to discard

3.2.7 Score Points

3.2.7.1 Description and Priority

The system must add/subtract the points equal to or exceeding the user bid that said user won/lost each round. This is a high priority because the calculation of points is what determines when the user has won/lost the game, so the calculation must be accurate and consistent.

- Benefit: 9
- Penalty: 3
- Cost: 4
- Risk: 3

3.2.7.2 Stimulus/Response Sequences

Stimulus: A round ends.

Response: The system determines whether the user has reached their bid.

Stimulus: The user does not reach their bid.

Response: The user is not awarded the points for reaching the bid.

Stimulus: The user reaches their bid.

Response: The user is awarded the points for reaching the bid.

3.2.7.3 Functional Requirements

REQ 1 – The system will calculate whether the user has reached their bid or not automatically.
REQ 2 – The system will add any points earned by the user this round to the total score.

3.2.8 **Win/Lose Screen**

3.2.8.1 Description and Priority

The system must end the game for the user and display a win/lose screen when a player's score meets or exceeds the winning point value. This feature is necessary for the game to end and for the user to be able to start a new game without closing and reopening the application. This makes the feature high priority.

- Benefit: 9
- Penalty: 7
- Cost: 5
- Risk: 3

3.2.8.2 Stimulus/Response Sequences

Stimulus: The target score is reached.

Response: The game is ended, and the system determines whether the user has won or lost.

Stimulus: The user has lost the game.

Response: The lose screen is displayed to the user.

Stimulus: The user has won the game.

Response: The win screen is displayed to the user.

3.2.8.3 Functional Requirements

REQ 1 – The system will automatically end the game once the target score selected by the user has been reached.

REQ 2 – The system will check after each round to see if the target score has been reached.

REQ 3 – The user will be sent to the win/lose screen depending on the results.

3.2.9 Dealing Cards

3.2.9.1 Description and Priority

For this functional requirement, the system must deal the correct number of cards (6 cards) to each player at the start of each round and if a player discards non-trump suit cards from their hand before the first round. Considering this feature is necessary to make the game playable; it is of high priority (9). Without this feature, the user and other players won't have any cards to play the game with, making this feature highly beneficial regardless of the cost.

3.2.9.2 Stimulus/Response Sequences

Stimulus: A new game is generated

Response: The system provides each player with the correct number of shuffled cards (6 cards)

Stimulus: A player discards an undetermined number of non-trump suit cards before round begins

Response: The system calculates the number of cards that each player discarded and provides them with that same number of new, shuffled cards.

3.2.9.3 Functional Requirements

REQ – 9: In order for the user to be able to utilize this feature, the "Start New Game" functional requirement must be implemented first in the software. If a new game hasn't been generated, then the user and the computer-generated bots acting as the other players aren't active participates to deal cards to in the first place.

There are no inputs required from the user for this feature to work. However, if this feature does encounter an error condition, such as dealing the wrong number of cards, the system should detect an error in the final calculation and restart the game.

3.2.10 Start New Game

3.2.10.1 Description and Priority

For this functional requirement, the system must allow the user to be able to start a new game when first loaded in and after the current game ends. Considering this feature is necessary to even begin playing the game; it is of high priority (9). Without this feature, the user won't be able to start a game to even play, making this feature highly beneficial regardless of the cost.

3.2.10.2 Stimulus/Response Sequences

Stimulus: The user loads into the game

Response: The system displays the new game button

Stimulus: The user finishes a game (both win or lose)

Response: The system displays the new game button

Stimulus: The user clicks the new game button

Response: The system generates a new game (removes all data from the previous game if any)

3.2.10.3 Functional Requirements

Considering this functional requirement generates a new game for the user to play in the first place and redirects them to the game interface, there are no functional requirements that must be implemented first in the software. None of the other features can work without the game's interface being visible and accessible.

While there is no typing inputs required from the user for this feature to work, the user must click the new game button to activate this feature. Therefore, no input can be considered invalid. However, if this feature does encounter an error condition, such as not generating a new game when the button is pressed, the system should detect an error in there being a lack of response to the input being entered (button pressed) and restart the system.

3.2.11 AI Logic for Computer-Generated Players

3.2.11.1 Description and Priority

For this functional requirement, the system must contain basic AI logic for the bots in order for them to both be able to participate in the game in the first place and have the ability to make rational decisions so the game can have a challenging element for the user. Considering this feature is necessary to make the game enjoyable; it is of high priority (8). Without this feature, the user won't have any competent players to compete against, making this feature highly beneficial regardless of the cost.

3.2.11.2 Stimulus/Response Sequences

Stimulus: The bot has the first turn in the trick

Response: The system will have the bot play the first card in their hand

Stimulus: It's the bot's turn in the trick and they are not first

Response: The system will find all the cards in the bot's hand that are valid to play, record all the cards already played during that trick, and determine which card to play depending on whenever or not they can win that trick.

3.2.11.3 Functional Requirements

REQ – 1: The system will select the card that the bots will play each sub-round (trick) depending on the valid cards in the bot's hand and the cards currently in the discard pile for that trick. For that to happen, the logic for the card selection feature must be operational.

There are no inputs required from the user for this feature to work. However, if this feature does encounter an error condition, such as the bots having no card in their hands during their turn, the system should detect an error when queuing for a card to play and skip their turn.

3.3 Use Cases

3.3.1 Use Case #1

Use Case Name	**FR-01-Card Selection**
Description	The application shall provide a graphical user interface that allows the player to select a single card from their current hand and play it onto the game board during their designated turn in each round.
Actors	User (Player)
Precondition	The user has started a game
Basic Path	1. The system highlights the user's hand to indicate it is their turn. 2. The user selects a valid card from their hand. 3. The system validates the card play (checks if it follows the lead suit if required). 4. The system moves the card to the "table" area and ends the user's turn. 5. The system triggers the next player's (bot) action.
Postcondition	The user´s turn will have finished
Exception Paths	• *Invalid Card:* If the user selects a card that does not follow the lead suit (and they have one), the system shall show a warning and prevent the play.

	• *Timeout:* If the countdown reaches 0, the system automatically plays the lowest value valid card for the user.
Other	

Table 1: Use case 1

3.3.2 Use Case #2

Use Case Name	**FR-02-Bid Choosing**
Description	The system shall display a bidding interface at the start of each round, allowing the user to select an initial bid amount within the range of 0 to 18 points.
Actors	User (Player)
Precondition	The user has started a game
Basic Path	• The system displays the bidding interface with options from 0 (Pass) to 18. • The user selects a bid amount higher than the current high bid. • The system records the bid and moves the turn to the next player.
Postcondition	The user has selected his bid for the round
Exception Paths	• *Lower Bid:* If the user tries to select a bid lower than or equal to the current high bid, the system disables those options or shows an error. • *Timeout:* If the countdown reaches 0, the system automatically registers a "Pass" (0 points) for the user.
Other	

Table 2: Use case 2

3.3.3 Use Case #3

Use Case Name	**FR-03-Choose Suit**
Description	The system must allow the user to be able to choose the trump suit for a round if they win the bid.
Actors	User (Player)
Precondition	The user has made a bid and won it
Basic Path	• The system prompts the user to select one of the four suits (Hearts, Diamonds, Clubs, Spades). • The user selects a suit. • The system sets the "Trump" for the remainder of the round. • The system enables the "Discard" phase.
Postcondition	The user has chosen the trump
Exception Paths	*Timeout:* If the countdown reaches 0, the system automatically selects the suit of the highest-value card in the user's hand as the trump.
Other	

Table 3: Use case 3

3.3.4 Use Case #4

Use Case Name	**FR-04-New Game**
Description	The system must allow the user to be able to start a new game when first loaded in and after the current game ends.
Actors	User (Player)

Precondition	The user has loaded into the web application "Pitch Deck" or has ended a game
Basic Path	• The user has loaded into the web application "Pitch Deck" and is directed to the home screen • The user clicks the "New Game" button • The system creates a new instance of the game
Postcondition	The user has started a new game of "Pitch Deck"
Exception Paths	*Finished Game:* The user has finished their current game and is offered the option of starting a new game which will create a new instance of the game. *Canceled Game:* The user has canceled out of their current game and is redirected to the home screen where they have the option of starting a new game which will create a new instance of the game (includes FR-05-Leave Game).
Other	

3.3.5 Use Case #5

Use Case Name	**FR-05-Leave Game**
Description	The system must allow the user to be able to leave the game before it ends, even if they are in the middle of a round, or after it ends.
Actors	User (Player)
Precondition	The user is currently in an active game or just finished a game
Basic Path	• The user has started a new instance of the game (includes FR-04-New Game) • The user clicks the "Leave Game" button • The user is redirected to the home screen of the application
Postcondition	The user has left their current game of "Pitch Deck"
Exception Paths	*Finished Game:* The user has finished their current game and is offered the option of leaving the game which will redirect the user back to the home screen.
Other	

3.3.6 Use Case #6

Use Case Name	**FR-06-Total Points**
Description	The system must require the user to choose the total points the game will go to.
Actors	User (Player)
Precondition	The user wants to start a game.
Basic Path	• The user has pressed start game • The system has the user select what they would like to play till (50, 75, 100, 150 total points) • The user selects the total points they play till • The system starts the game
Postcondition	The user will then start a game.
Exception Paths	*Invalid input:* The user tries to start a game without inputting a total. This will result in an error.
Other	

Table 6: Use case 6

3.3.7 Use Case #7

Use Case Name	**FR-07-View Scoreboard**

Description	The system will display the number of points each player has obtained during the game. The score will be updated after each round.
Actors	User (Player)
Precondition	The user has started the game.
Basic Path	• The system shows the players score during the game/after each round • The score is updated after each round
Postcondition	The user is allowed to see accurate game scores, while not being allowed to edit it.
Exception Paths	*Error:* A score update could not be processed. A failure error will appear, notifying the player. It will also keep the previous score. *Modification:* The user attempts to change the score. The system will not allow this. If it does occur, the game will automatically end.
Other	

Table 7: Use case 7

3.3.8 Use Case #8

Use Case Name	**FR-08-Card Lock**
Description	The system allows the user to play the correct cards at the right time. This ensures correct game play.
Actors	User (Player)
Precondition	The user has started the game, and bidding has been completed.
Basic Path	• The system shows that it is the users turn • The user's hand is shown • The system shows the user which cards they are allowed to play • Player selects a valid card • Round proceeds
Postcondition	• The selected card is removed from the user's hand • The selected card is played/put into middle for all players to see • Game proceeds to next step
Exception Paths	*Error:* The user tries to throw a card that does not follow suit. As a result, an error notification will appear, explaining why the user cannot use that card. *Error 2:* The user tries to throw it out of turn. This all results in a notification stating the user must wait for their turn. *Error 3:* During the throw in, non-trump cards must be discarded. If the user tries to discard trump cards, an error will appear informing the user that they cannot do this.
Other	

Table 8: Use case 8

3.3.9 Sub-use Case #1

Use Case Name	**FR-09-Score Points**
Description	The system will automatically determine whether each player reached their bid at the end of each round.
Actors	User (Player)
Precondition	The user has started the game and a round has completed with bids and trick totals recorded.
Basic Path	• The round ends • The system retrieves each player's bid for the round

	• The system retrieves each player's actual tricks/wins for the round • The system compares actual results to bids • The system stores whether the bid was met for each player
Postcondition	The system has an accurate "bid met / bid not met" result stored for each player for scoring.
Exception Paths	*Error*: Bid or round results are missing. A failure error will appear and scoring will not occur until the required values are available. *Modification*: The user attempts to change bid results after the round ends. The system will not allow this.
Other	

Table 9: Sub-use case 1

3.3.10 Sub-use Case #2

Use Case Name	**FR-10-Win/Lose Screen**
Description	The system will display a win/lose screen when the game ends, based on whether the user won or lost. The screen will show final scores and allow the user to start a new game.
Actors	User (Player)
Precondition	The game has ended because the target score was reached.
Basic Path	• The game ends • The system determines whether the user won or lost • The system displays the appropriate win or lose screen • The system displays the final scores • The user is given the option to start a new game
Postcondition	The user is shown the game outcome and can restart without closing the application.
Exception Paths	*Error*: The system cannot determine the outcome. A failure error will appear and the system will display the final scores only. *Modification*: The user attempts to return to the game board after the game ends. The system will not allow this and will keep the user on the results screen.
Other	

Table 10: Sub-use case 2

3.4 Non-Functional Requirements

NFR 1 – The system will allow the user to choose/play a card. These user actions will be processed in <= 1 second for 95% of transactions.

NFR 2 – The system must support at least 4 players

NFR 3 – The scoring will be processed after each round, resulting in the scoreboard being updated. This will occur within <= 1 second for 90% of transactions.

NFR 4 – They system will allow a user to start/exist a game within 1 minute.

NFR 5 – The system will display an error notification for an invalid action within <= 2 seconds for 95% of transactions.

NFR 6 – They system will have visuals for valid cards, and when it is the players turn.

NFR 7 – Users will not be able to change rules or scores.

3.5.1 Performance

The system for the game Pitch Deck will support user actions in a timely manner, ensuring a fun and fast gaming environment. User actions, such as choosing and playing a card will need to be processed in <= 1 second, for 95%

of transactions. Scoring should be updated within that same timespan for 90% of transactions after each round. Error notifications should appear quickly, when the user is attempting an invalid move. The user should also be allowed to start/leave a game with no major delay.

3.5.2 Reliability

Pitch Deck will be a consistent, reliable game. It will have the same behaviors for each round, not allowing users to make edits to the rules. Scoring logic will remain the same and remain unmodified. If an error occurs, the game will maintain integrity by not allowing that round to count and keep the pervious score. These reliability rules will be enforced throughout the system.

3.5.3 Availability

The availability of the **Pitch** application is primarily dependent on the web hosting service and the user's client-side environment.

- **Hosting Uptime:** The application shall be hosted on a reliable static hosting platform, aiming for an uptime of 99.9% to ensure the game is accessible to users at any time.
- **Offline Accessibility:** Once the initial web assets (HTML, CSS, JS) are loaded into the browser, the application shall remain functional even if the internet connection is lost, as all game logic and state management are handled locally.
- **State Recovery:** In the event of a browser crash or accidental page refresh, the system shall utilize Local Storage to attempt to recover the current game state, ensuring the user does not lose their progress in the middle of a round.

3.5.4 Security

Given that the application is a single-player, serverless game with no user accounts or backend database, security requirements focus on client-side integrity and data privacy.

- **Data Privacy:** The application shall not collect, store, or transmit any Personally Identifiable Information All user preferences and game scores are to be stored strictly within the user's local browser environment.
- **Input Validation:** The system shall validate all user inputs to prevent any potential script injection or cross-site scripting (XSS) attacks that could compromise the local browser session.
- **Integrity:** While the game is for entertainment purposes, the system shall ensure that game state variables (scores, card values) are managed in a way that prevents accidental corruption of data within the Local Storage.
- **No Authentication:** As defined in the Project Charter, the system will not require a login or password, eliminating risks associated with credential theft or database breaches.

3.5.5 Maintainability

The Pitch Deck application is a single-player, serverless game that will not require any maintenance. Updates could be made alongside OS updates to ensure optimal performance; however, this would be optional. Other than OS updates, there is no other maintenance because the application will follow the same behaviors every time.

3.5.6 Portability

The portability of the Pitch Deck application will be primarily dependent on the operating system of the user. If the user is running on the required OS or newer, the application will work. The application will have consistent performance and functionality across all platforms on which it is available. The application does not require any high-performance devices and will run on any device with basic specs.

3.5 Design Constraints

Our version of the card game Pitch is a web application that will only be able to be accessed using a web browser. This application will contain no authorization or login system, meaning that a user's data, including their history, past scores, and any identification, will not be saved in any external database. Since this application will use local storage,

there will be no way for users to be able to differentiate themselves, nor will the system keep track and save their progress. This system will not require any additional hardware pieces to run; therefore, it will be readily accessible on computers, laptops, and other mobile devices. Lastly, the development of the software, including the backend and frontend, will be done using basic technologies such as Flask and GitHub which will limit the creation and implementation of more complicated features.

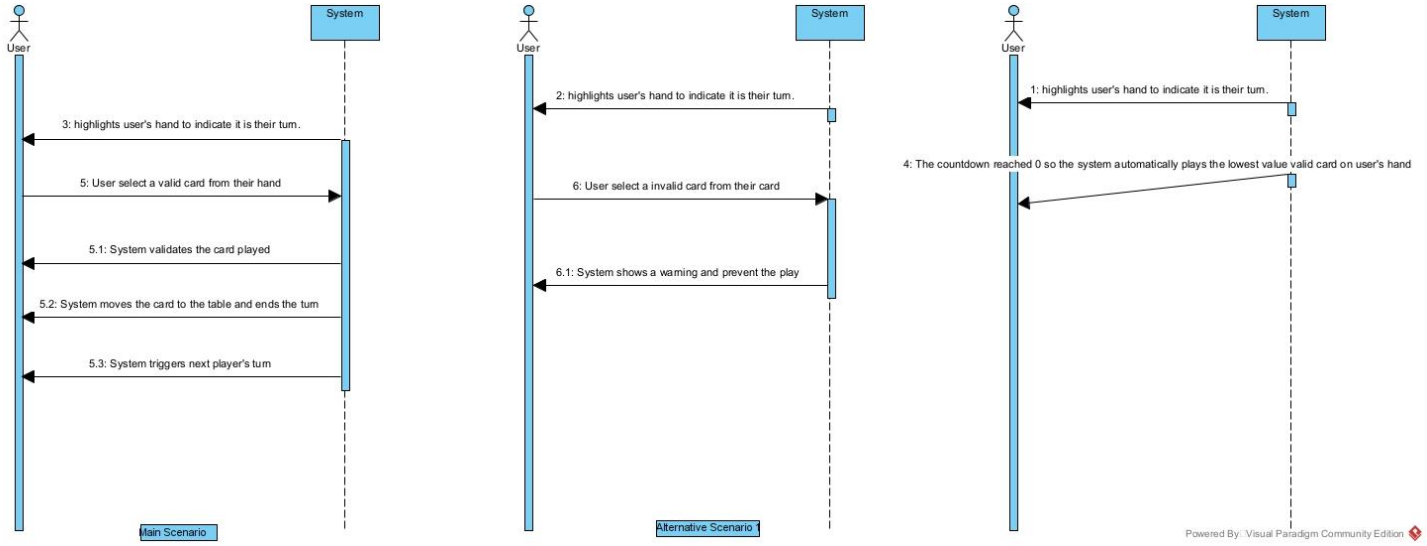
3.6 Logical Database Requirements

Our virtual card game, Pitch Deck, is a server-less web application that will have no multiplayer option where the user will have the ability to join other online players to compete against them in real time and no authentication or login system. Because this application lacks those two specific features, there is no need to save and store any user information such as usernames and scoring history. Each game will be its own instance and will not depend on information from past games. Considering all these factors, there is no need for the integration of an external database. This application will use local storage or browser-based storage that will store information for each session, but nothing long-term. No implementation of a backend database will be required.

4. Analysis Models

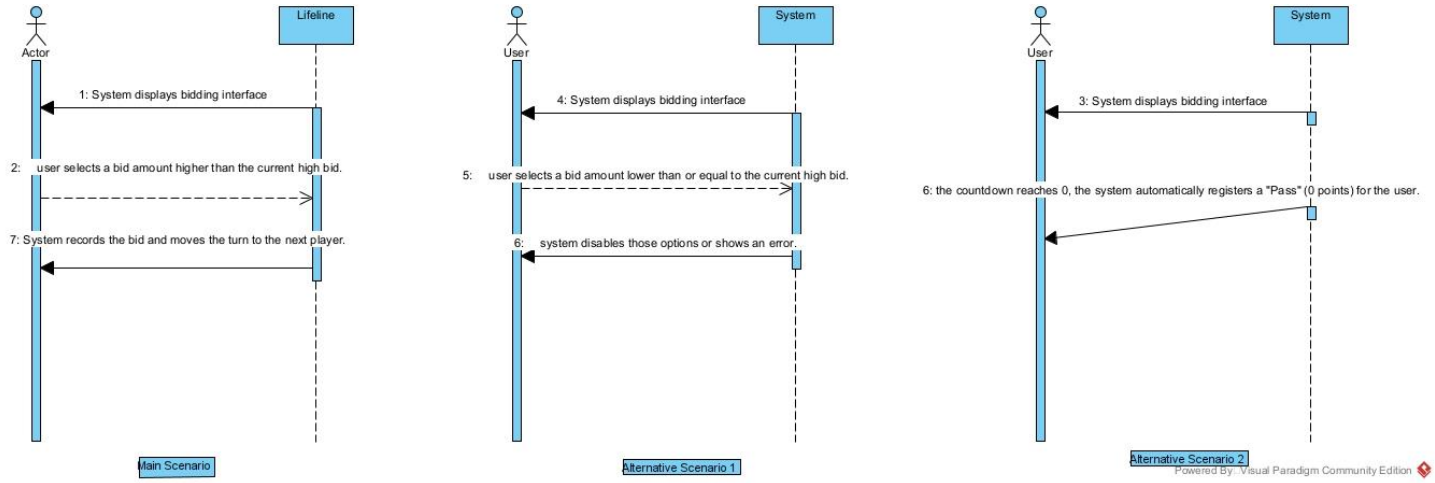
3.1 Sequence Diagrams

System Sequence Diagram for Use Case: **Choose Card**

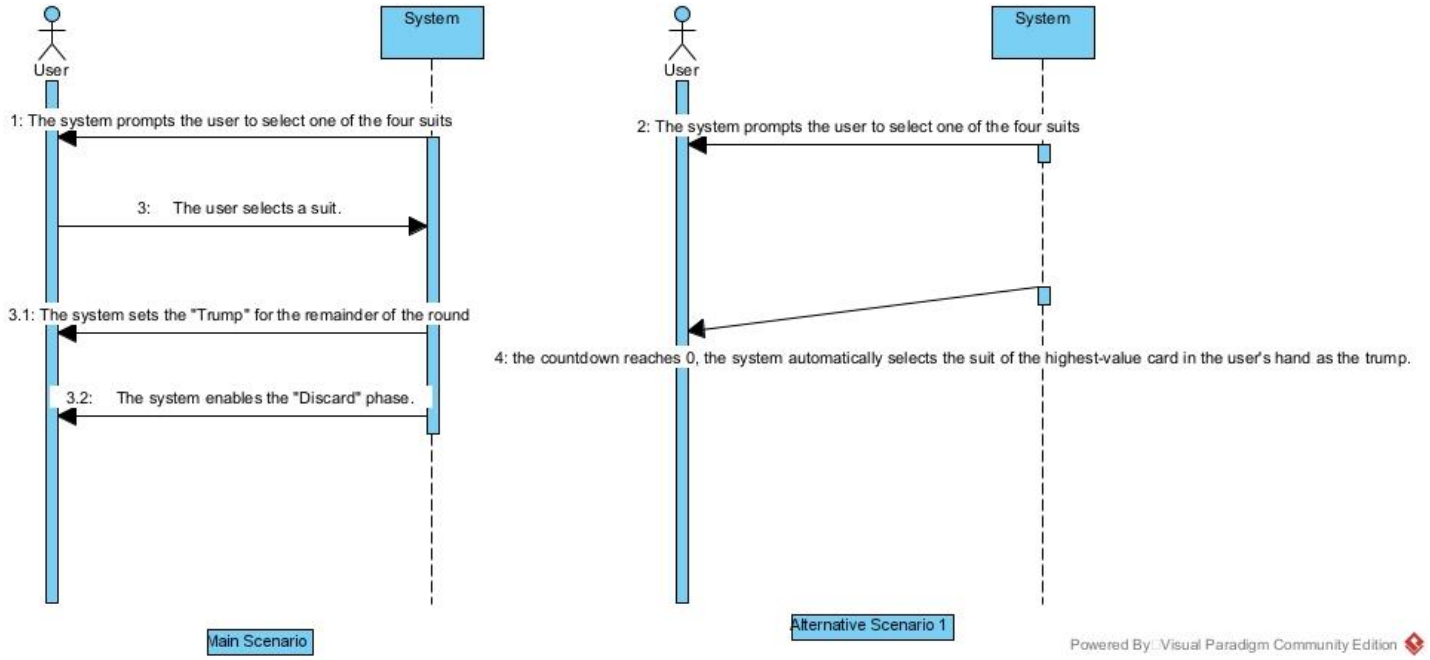


System Sequence Diagram for Use Case: **Choose Bid**

Pitch Deck

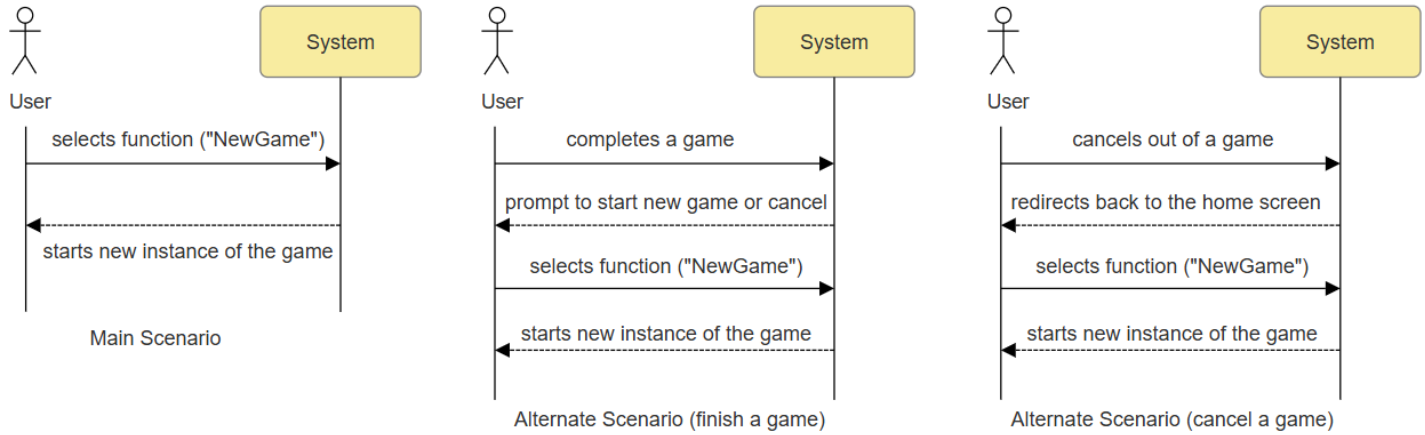


System Sequence Diagram for Use Case: **Choose Suit**

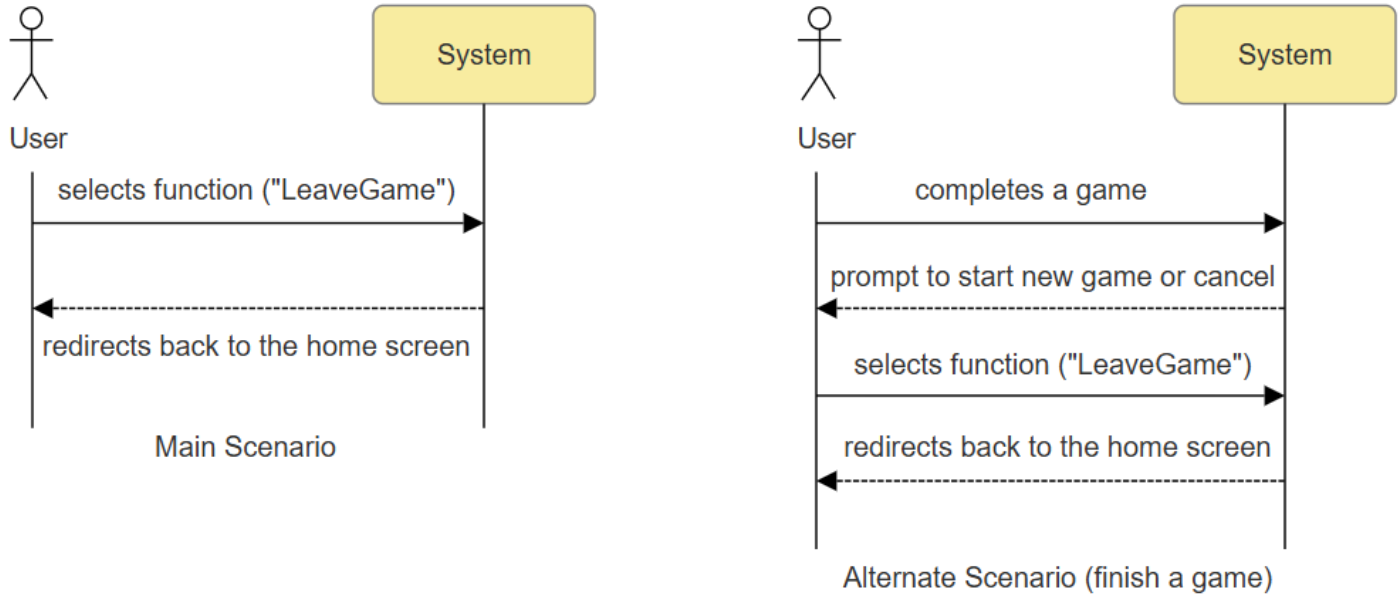


System Sequence Diagram for Use Case: **New Game**

Pitch Deck

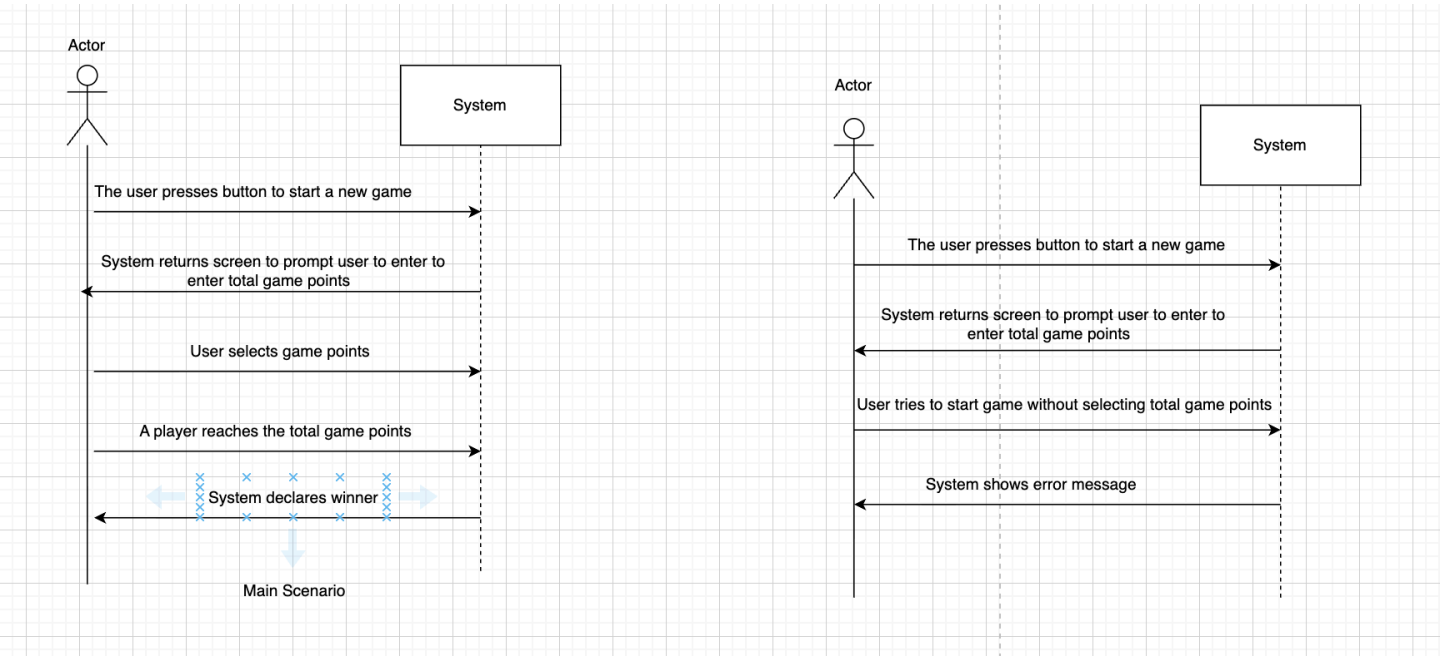


System Sequence Diagram for Use Case: **Leave Game**

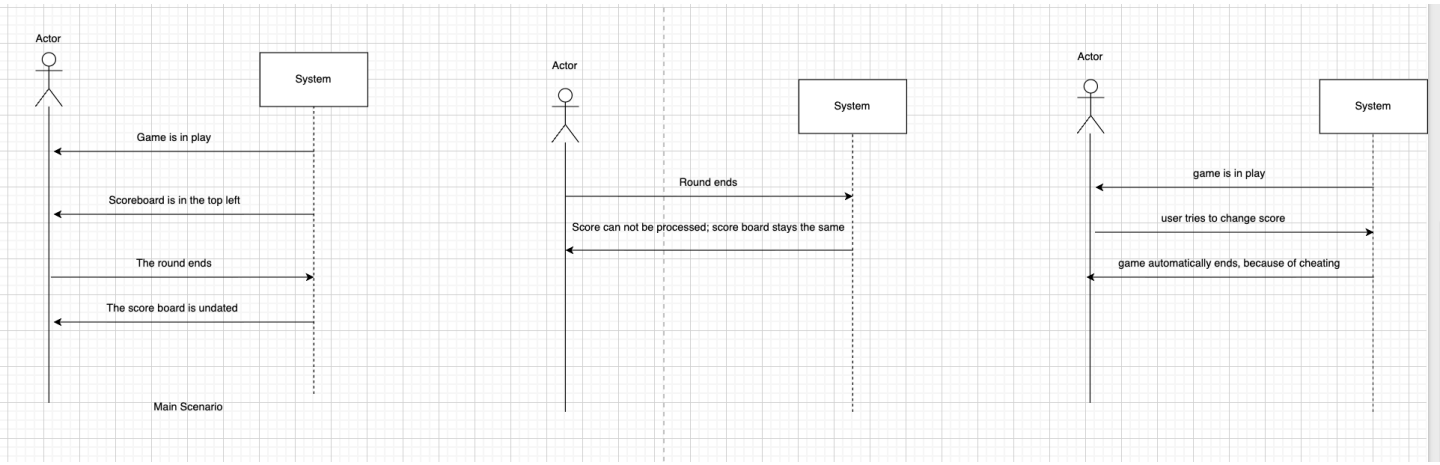


System Sequence Diagram for Use Case: **Total Game Points**

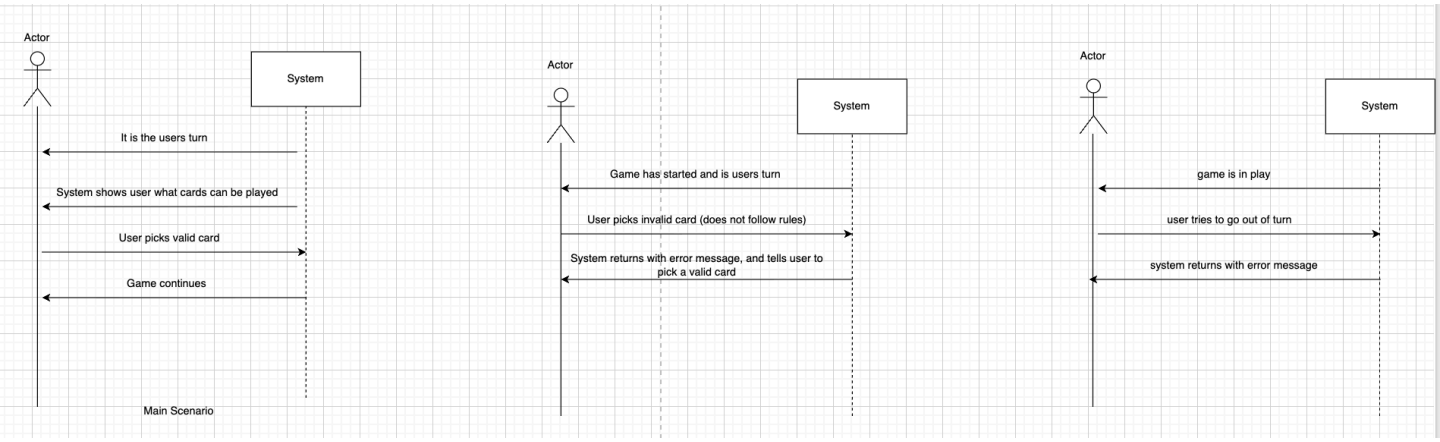
Pitch Deck



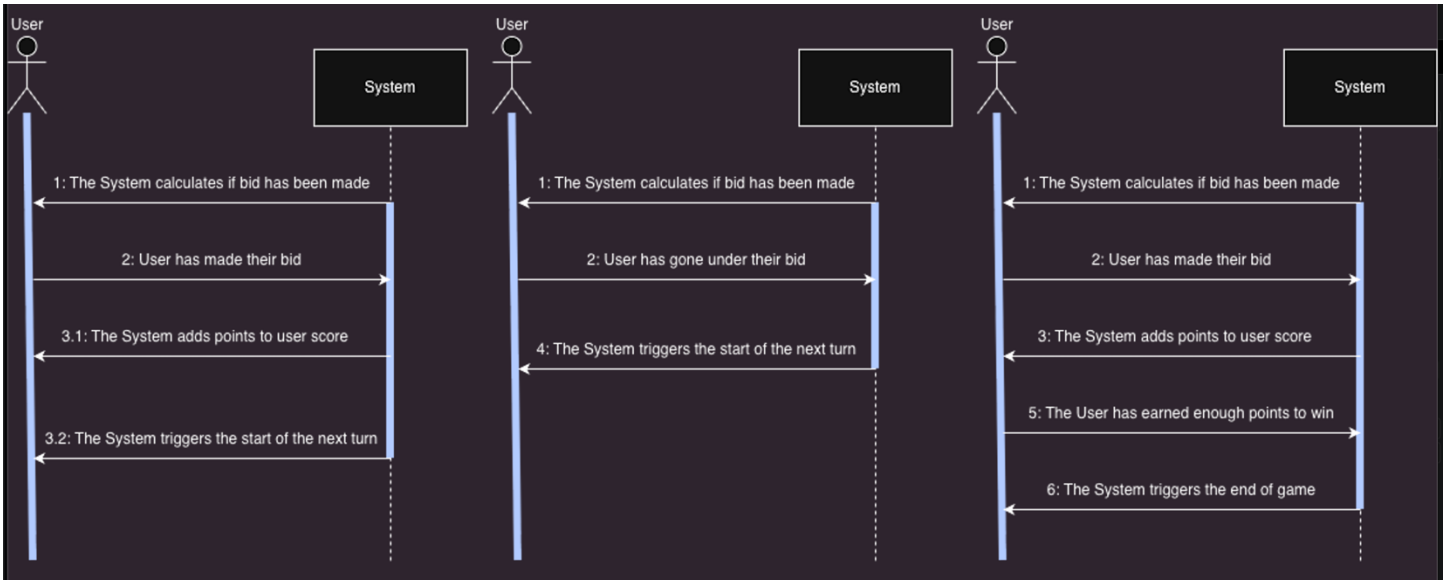
System Sequence Diagram for Use Case: **View Scoreboard**



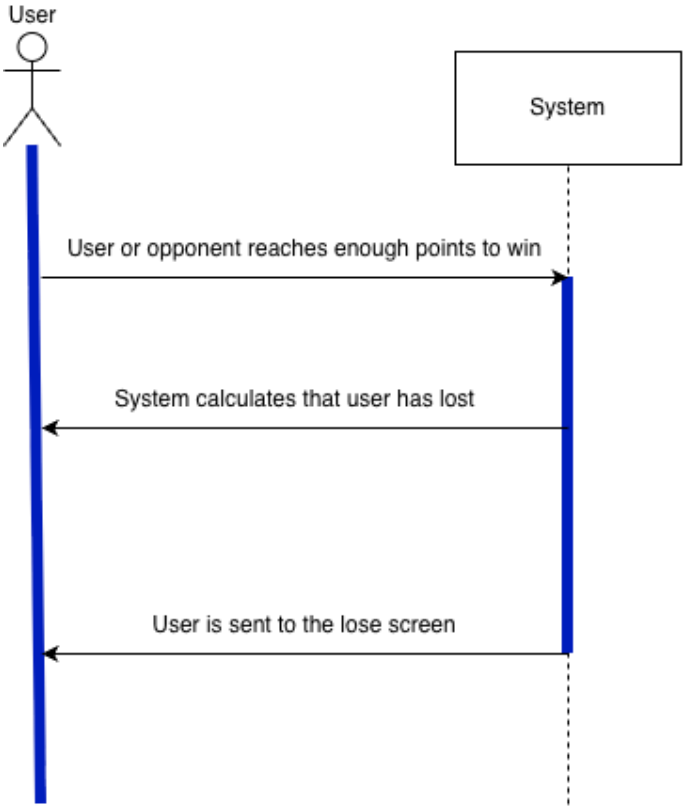
System Sequence Diagram for Use Case: **Card Lock**

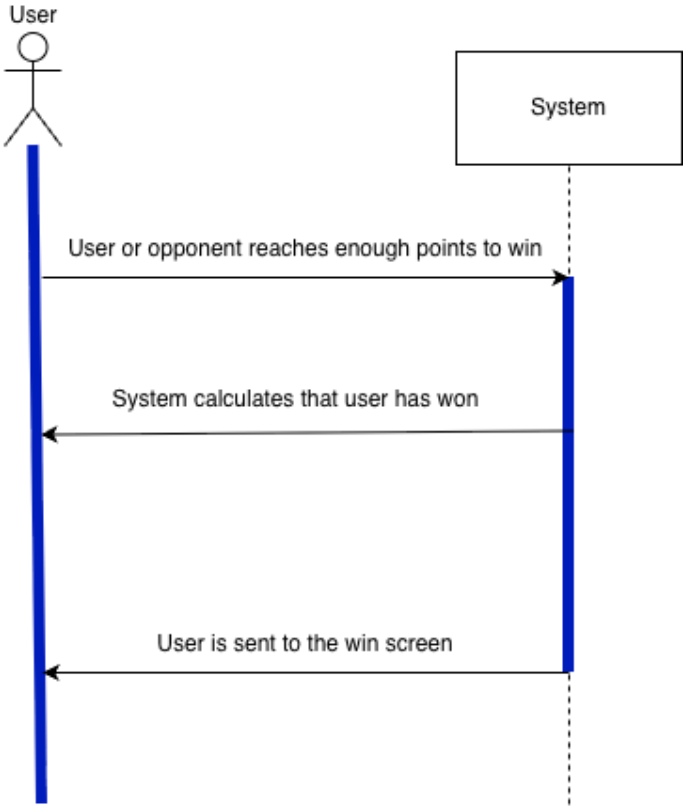


System Sequence Diagram for Sub Use Case: Score Points



System Sequence Diagram for Sub Use Case: Win/Lose Screen





5. Change Management Process

The SRS is a live document, meaning that it will be updated, when necessary, throughout the project's development. Whenever the SRS is updated, the changes will be submitted by the current team leader. Before the document is updated, the concepts that are to be added or changed about the document must have unanimous approval by all team members first and then unanimous approval after the concepts are added or changed to the document. After the document has been successfully updated, the team leader will add the changes to the revision history at the beginning of the document and include information such as when the update has been completed and approved, the new version number of the document, who the author is, and a description about what the changes were. After the history has been updated, the other team members must sign off that they have given approval to these changes in the document approval section where they will include their signature, their printed name, their title in the project, and the date of when the update has been completed and approved by them.

References

A. Appendices

A.1 Appendix 1

A.2 Appendix 2